Excel for Finance

The Complete Professional Handbook

Module 1

Excel Foundations

Interface, Data & Number Formatting, Formula Fundamentals, Shortcuts — with Python and SQL Equivalents

Companion modules in this handbook:

- **Module 1** — Excel Foundations *(this document)*
- **Module 2** — Making Excel Professional: Optimisation, Macros, Absolute/Relative References, Named Ranges, Data Validation & Custom Formatting
- **Module 3** — Excel Like a Pro: Conditional Formatting, Logical Functions, Lookup Functions (VLOOKUP/HLOOKUP/XLOOKUP/INDEX-MATCH), Conditional Aggregation, Error Handling, String Functions

Compiled study notes covering the Excel curriculum end-to-end, extended with equivalent Python (`pandas`, `openpyxl`) and SQL implementations of every concept, for a tech-background learner building cross-tool fluency in financial analytics.

August 15, 2026

Contents

1 How to Use This Handbook **2**

2 Why Excel Still Matters **2**

3 The Excel Environment **3**
3.1 Core Vocabulary . . . 3
3.2 The File Tab (Backstage View) . . . 4
3.3 The Home Tab . . . 5
3.3.1 Clipboard Operations . . . 6
3.3.2 Font, Colour & Alignment . . . 6
3.3.3 Wrap Text and Merge & Center . . . 7
3.4 The Insert Tab . . . 7
3.5 The View Tab . . . 8

4 Data Types and Number Formatting **8**
4.1 Built-in Number Formats . . . 9
4.2 Custom Number Format Codes . . . 9
4.3 Text Numbers: A Silent Data-Quality Trap . . . 11

5 Formula Fundamentals **11**
5.1 Anatomy of a Formula . . . 11
5.2 Cell References — A First Look . . . 11
5.3 Starter Functions Every Analyst Uses Daily . . . 12
5.4 Seeing the Formulas, Not Just the Results . . . 13

6 Keyboard Shortcuts for Speed **13**
6.1 Navigation . . . 13
6.2 Selection . . . 13
6.3 Editing . . . 14
6.4 Formatting . . . 14
6.5 Formulas & Calculation . . . 14
6.6 File & View . . . 14

7 Cross-Tool Translation: Python **14**
7.1 Why Two Libraries, Not One . . . 15
7.2 Workbook and Worksheet Equivalents (openpyxl) . . . 15
7.3 Reading a Whole Sheet as a DataFrame (pandas) . . . 15
7.4 Number Formatting (openpyxl) . . . 16
7.5 Aggregate Formulas: pandas vs. Writing Live Excel Formulas . . . 16
7.6 Saving and Exporting . . . 17

8 Cross-Tool Translation: SQL **17**
8.1 The Mental Model . . . 17
8.2 Defining the Table (equivalent of setting up columns & formats) . . . 18
8.3 Inserting Data (equivalent of typing values into cells) . . . 18
8.4 Aggregate Queries (equivalent of SUM, AVERAGE, COUNT, MIN, MAX) . . . 18

9 Worked Example: Monthly Expense Summary, Three Ways **19**
9.1 The Data . . . 19
9.2 Excel Implementation . . . 19
9.3 Python Implementation . . . 19
9.4 SQL Implementation . . . 20

10 Common Pitfalls & Debugging Checklist **21**

11 Chapter Summary & Self-Check **21**

12 What's Next: Module 2 Preview **22**

1 How to Use This Handbook

This handbook is written for someone who already thinks like a programmer but needs to become fluent in Excel because it remains the lingua franca of the finance industry. Every topic is therefore explained three times, in three dialects:

- **Excel** — the native implementation, exactly as you would build it in a worksheet, including menu paths and keyboard shortcuts.
- **Python** — the equivalent using `pandas` (for tabular data manipulation) and `openpyxl` (for reading, writing, and formatting actual `.xlsx` files).
- **SQL** — the equivalent using standard SQL (schema design, `SELECT`/`INSERT` statements, and aggregate queries), since a worksheet is conceptually a table and a workbook is conceptually a small database.

Throughout the document you will see the following recurring boxes:

Key Concept

A foundational idea you must internalise before moving on — these are the "why", not just the "how".

Finance Application

How the concept is specifically used in financial modeling, accounting, or investment analysis — because a generic Excel tutorial and a *finance* Excel tutorial diverge here.

Pro Tip

A workflow habit that separates a fast, professional analyst from a slow, error-prone one.

Common Pitfall

A mistake that looks harmless but silently corrupts a financial model — these are worth memorising even more than the formulas themselves.

Formula Syntax

The precise syntax of a function or feature, written the way Excel's own formula tooltip would write it.

2 Why Excel Still Matters

It is a reasonable instinct, coming from a technical background, to see Excel as a legacy tool that Python (`pandas`, `numpy`), SQL, and BI platforms like Power BI have functionally superseded. In pure data-processing terms, that instinct is often correct: Excel does not version-control well, struggles past a few hundred thousand rows, has no real automated testing story, and its formulas are far more error-prone at scale than equivalent code. Studies of real-world spreadsheets consistently find silent formula errors in a meaningful fraction of production financial models.

However, Excel persists as the dominant interface of finance for structural reasons that have nothing to do with raw computational power:

1. **It is the shared language of non-technical stakeholders.** A CFO, an investment committee, or a client will open a workbook, not a Jupyter notebook or a SQL client. The deliverable of financial analysis is very often *the workbook itself.*

2. **It compresses the edit-see-verify loop to zero.** Changing an assumption and watching every downstream number recalculate instantly is a uniquely powerful mode of exploratory reasoning that scripted pipelines do not replicate as fluidly.
3. **It is auditable cell-by-cell.** Every number in a model can be traced to its exact formula and inputs by clicking on it (§5), which matters enormously in regulated, audit-heavy environments.
4. **It is the interview and on-the-job standard.** Investment banking, equity research, FP&A, and corporate finance roles test and expect Excel fluency directly, independent of whatever other tools a team may use internally.

The Right Mental Model

Excel, Python, and SQL are not competitors for the same job; they are three tools with different strengths that a modern finance professional moves between deliberately:

- **SQL** owns *storage and retrieval* — pulling the right slice of data out of a large, governed source system (a general ledger, a market data warehouse, a trading system).
- **Python** owns *scale, automation, and repeatability* — cleaning, transforming, and modeling data too large or too repetitive to do by hand, and building pipelines that must run unattended.
- **Excel** owns *presentation, exploration, and communication* — the last mile where a human needs to see, sanity-check, adjust, and present the numbers.

A professional workflow frequently chains all three: SQL pulls data from a database → Python cleans and models it → Excel presents and lets a decision-maker interact with it. This handbook teaches Excel deeply while keeping that chain visible, so every Excel concept is anchored to its Python/SQL counterpart rather than learned in isolation.

3 The Excel Environment

3.1 Core Vocabulary

Before anything else, four terms need to be precise, because they map directly onto concepts you already know from databases and dataframes.

Excel term	Definition	pandas equiv.	SQL equiv.
Workbook	The entire `.xlsx` file; a container for one or more worksheets.	An Excel `.xlsx` file opened with several sheets (a "Book" in `openpyxl`)	A **database**
Worksheet (Sheet)	A single grid within a workbook, referenced by a tab at the bottom.	A single `DataFrame`	A **table**
Cell	The intersection of one column and one row; holds one value or formula.	One element of a DataFrame, `df.iloc[r, c]`	One **field** in one **row**
Cell Address	The unique identifier of a cell: **Column letter** + **Row number**, e.g. `C18`.	`df.iloc[17, 2]` (0-indexed)	Not applicable — rows are unordered by design

Key Concept

A cell address like `C18` is a *positional* reference — it says "third column, eighteenth row," not "the revenue cell." This is fundamentally different from a SQL column, which is always referenced by *name* (`revenue`), never by position. This one difference explains why Excel formulas break when rows are inserted/deleted while SQL queries never do — and it is the entire motivation for Named Ranges, covered in Module 2.

Other vocabulary you will see constantly:

- **Range** — a rectangular block of cells, written as `TopLeft:BottomRight`, e.g. `B4:B8` (a column of 5 cells) or `A1:D10` (a 4-column, 10-row block).
- **Ribbon** — the tabbed toolbar at the top of the window (**File, Home, Insert, Page Layout, Formulas, Data, Review, View**, plus the **Developer** tab once enabled for macros in Module 2).
- **Formula Bar** — the input line just below the Ribbon that shows the raw formula/content of the currently selected cell (as opposed to its displayed, formatted value).
- **Name Box** — the small field to the left of the Formula Bar showing the active cell's address (or its name, once Named Ranges are introduced).

3.2 The File Tab (Backstage View)

The **File** tab is not part of the worksheet grid at all — it is Excel's administrative control panel, equivalent to a file-system/session-management layer sitting outside your "data" layer.

Command	What it does
New	Create a **Blank Workbook**, or start from a **Template** (amortisation schedules, budget trackers, invoice templates, calendars) that ships pre-built formulas and formatting.
Open	Load an existing workbook from local disk, a network drive, or cloud storage (OneDrive/SharePoint), via **Browse**.
Save (`Ctrl+S`)	Overwrites the current file in place, in its current format, at its current path.
Save As (`F12`)	Writes a *new* file — a different name, a different folder, and/or a different **format**.
Print (`Ctrl+P`)	Opens the print preview & setup pane (page orientation, scaling to fit, print area).
Export	Converts the workbook to another format without a full "Save As" dialog, most commonly **PDF**.

File Formats a Finance Analyst Actually Uses

- `.xlsx` — the standard working format; formulas, formatting, and macros(-enabled variant `.xlsm`) are preserved.
- `.xls` — the legacy pre-2007 binary format; you will still meet this from older systems or bank exports.
- `.csv` — plain comma-separated *values only*: no formulas, no formatting, no multiple sheets. This is almost always the format used to move data *between* Excel, Python, and SQL, because every tool can read it.
- **PDF export** — used when *sending* a finished model or report to someone who should see the numbers but not be able to edit formulas or see underlying assumptions (e.g., a distributed investor report).

Saving a Formula-Driven Model as CSV

CSV format stores only the currently *displayed values*, not formulas, named ranges, formatting, or multiple sheets. Saving a multi-tab financial model as `.csv` silently destroys everything except one sheet's raw numbers. Always keep the working file as `.xlsx`, and only export to CSV as a final, disposable "data handoff" step.

3.3 The Home Tab

The Home tab holds the tools you touch most often: moving data around, and controlling how it looks.

3.3.1 Clipboard Operations

Action	Shortcut	Behaviour
Cut	`Ctrl+X`	Removes the data from its original location once pasted elsewhere; any formula referencing that cell is automatically updated to point to the new location.
Copy	`Ctrl+C`	Duplicates the data; the original is left untouched.
Paste	`Ctrl+V`	Inserts the cut/copied content at the new location.
Format Painter	(Home tab icon)	Copies *only the visual formatting* of a cell (font, colour, borders, number format) — not its value or formula — and applies it elsewhere. Double-click the icon to keep it "on" and paint multiple ranges in a row.

Key Concept

"Paste" in Excel is not one operation but a family of them, accessible via `Ctrl+Alt+V` (*Paste Special*): **Paste Values** (numbers only, formulas discarded — essential for "freezing" a calculation), **Paste Formulas**, **Paste Formats**, and **Transpose** (flip rows into columns). Financial analysts use Paste Values constantly to convert a formula-derived number into a static input before sharing or archiving a model.

3.3.2 Font, Colour & Alignment

- **Font/size** — typeface (commonly *Calibri* or *Arial* in corporate templates) and size, changeable via dropdown or the Increase/Decrease Font Size buttons.
- **Bold / Italic / Underline** — standard emphasis, used in models to distinguish section headers, subtotal rows, and key outputs.
- **Font Colour vs. Fill Colour** — Font Colour changes the text hue; Fill Colour shades the entire cell background. In modeling, colour is not decorative — it is a coding convention (see the box below).
- **Alignment** — horizontal (left/centre/right), vertical (top/middle/bottom, useful once row height is increased), and **Orientation** (rotate text diagonally/vertically, common for narrow date-column headers).
- **Indentation** — increases the gap between the cell border and the text; used heavily to create visual hierarchy in a chart of accounts (e.g., indenting sub-line-items under a "Total Revenue" header).

The Standard Financial-Modeling Colour Convention

Wall Street and most professional FP&A teams follow an unwritten but near-universal rule:

- **Blue font** = a hard-coded input (a number someone typed in directly, not calculated).
- **Black font** = a formula that references *other cells on the same sheet.*
- **Green font** = a formula that pulls from *another sheet or another workbook.*
- **Red font** (or red parentheses via number formatting, §4) = a negative value, especially outflows/losses.

This lets anyone open your model, glance at the colours, and instantly know which cells are safe to overwrite (blue) and which are load-bearing calculations (black/green) — without reading a single formula. Adopt this from day one; it costs nothing and instantly signals professionalism.

3.3.3 Wrap Text and Merge & Center

Wrap Text forces long text content to stay visually inside one cell by increasing the row's height and flowing the text onto multiple lines, instead of letting it overflow visually into the next cell.

Merge & Center combines several selected cells into a single larger cell and centers its content — the classic use is creating a title banner across the top of a report (e.g., merging `A1:F1` for the title "Q3 2026 Income Statement").

Merged Cells Break Everything Downstream

Merged cells are one of the most common sources of broken financial models. Specifically, they cause:

- **Sorting failures** — Excel cannot sort a range containing merged cells of inconsistent sizes.
- **VLOOKUP/INDEX-MATCH errors** (Module 3) — a lookup formula expects one value per row; a merged cell only holds its value in the *top-left* cell of the merge, and every other cell in that merge reads as blank.
- **Copy/paste and filter misalignment** — filtering a table with merged header or body cells frequently shifts data unpredictably.

Professional convention: never merge cells inside a data table. Reserve merging strictly for standalone title banners above/outside the actual data range. To *visually* center a title over several columns without merging, use **Format Cells → Alignment → Horizontal: Center Across Selection** instead — it looks identical but keeps every underlying cell independently addressable.

3.4 The Insert Tab

The Insert tab adds elements that live *on top of* the grid rather than inside individual cells.

- **Illustrations** — Shapes, Pictures, Icons, 3D Models, Screenshots — used for annotating dashboards or building simple process diagrams directly on a sheet.
- **Text Box** — a floating box of text not tied to any cell; useful for callouts and commentary on a dashboard that must stay in a fixed position regardless of row/column resizing.
- **Charts** — converts a selected range into a visual. For a basic **2D Column/Bar Chart**, Excel maps the **X-axis** to categories (e.g., expense categories: Rent, Food, Transport) and the **Y-axis** to the numeric value (e.g., amount spent). Charting is covered in full depth in the Financial Functions/Charts module later in the course; here it is enough to recognise that a chart is simply a live *view* over a range — if the underlying cells change, the chart redraws automatically.

3.5 The View Tab

- **Headings / Gridlines / Formula Bar toggles** — hide or show the row/column labels, the light grey cell borders, and the formula input bar respectively; commonly turned off when preparing a clean, presentation-style dashboard.
- **Zoom** — scales the visual size of the sheet from a custom percentage, purely cosmetic and per-user (it does not affect print output unless separately configured).
- **Freeze Panes** — keeps specific rows/columns permanently visible while the rest of a large sheet scrolls underneath them. This is essential once a model or dataset exceeds one screen.

Freezing Panes — Step by Step

1. Click the cell *immediately below and to the right* of what you want frozen. (E.g., to freeze row 1 as a header and column A as labels, click cell `B2`.)
2. **View → Freeze Panes → Freeze Panes.**
3. Everything above and to the left of that cell now stays fixed as you scroll.
4. **View → Freeze Panes → Unfreeze Panes** reverses it.

Finance Application

Freeze Panes is used on almost every real financial dataset — e.g., freezing the top row of an annual report's line-item labels and the leftmost column of account names, so that as you scroll right through 10 years of quarterly columns, you never lose track of *which* account or *which* period you are looking at.

4 Data Types and Number Formatting

This is the single most under-appreciated topic in introductory Excel courses, and arguably the most important one for finance specifically. A number's *format* is entirely cosmetic — it changes only how a value is *displayed*, never the underlying value stored in the cell — but in finance, presentation *is* the deliverable, and misreading a formatted number (thousands vs. millions, or a positive read as negative) has real consequences.

Key Concept

Formatting and value are two independent layers in Excel. The cell literally stores `1234.5`; the *Number Format* property is a separate instruction telling Excel how to *render* that stored value as text on screen (e.g., as `$1,234.50`, as `1,234.5%`, or as `Jan-15`). This is directly analogous to the separation in a database between a column's stored `DECIMAL` value and the application/BI layer's display formatting (§8) — SQL never stores "$1,234.50" as such, only the number `1234.50`.

4.1 Built-in Number Formats

Format	Example display	Typical use
General	`1234.5`	Default; no consistent decimal alignment or separators — rarely acceptable in a finished model.
Number	`1,234.50`	Plain numeric values with a fixed decimal count and optional thousands separator.
Currency	`$1,234.50`	Currency symbol immediately adjacent to the digits; symbol position moves with column width.
Accounting	`$  1,234.50`	Currency symbol **left-aligned** at the cell edge, value right-aligned, decimal points vertically aligned down an entire column — the standard for financial statements.
Percentage	`12.35%`	Multiplies the stored value by 100 and appends `%` for display only (stored value stays a decimal fraction, e.g. `0.1235`).
Date/Time	`15-Aug-26`	Stored internally as a **serial number** (days since 1-Jan-1900) so dates can be added/subtracted arithmetically.
Text	`00123`	Forces the cell to treat entries literally as characters, never as a calculable number — used for IDs, account/routing numbers, phone numbers.

Currency vs. Accounting Format — They Are Not the Same

It is extremely common to confuse these two. **Currency** format glues the `$` sign to the first digit, so it drifts left/right as the value's digit-count changes — fine for a single number, ugly in a column. **Accounting** format pins the `$` sign to the far-left edge of the cell and right-aligns the number, so every dollar sign in a column lines up in one vertical strip and every decimal point lines up in another. Financial statements (income statements, balance sheets) almost always use **Accounting** format for exactly this reason — it is what makes a column of numbers scannable at a glance.

4.2 Custom Number Format Codes

Beyond the built-in presets, Excel lets you write your own format *code*, accessed via `Ctrl+1` → **Number** → **Custom**. This is a small, dense syntax worth learning properly.

Custom Format Code Structure

```
POSITIVE_FORMAT ; NEGATIVE_FORMAT ; ZERO_FORMAT ; TEXT_FORMAT
```

Up to four sections, separated by semicolons:

- **One section** → applied to all values (positive, negative, zero, text alike).
- **Two sections** → first applies to positive numbers *and* zero, second to negative numbers.
- **Three sections** → positive, negative, zero respectively (text falls back to General).
- **Four sections** → positive, negative, zero, text — full control over every case.

The most common building-block codes:

Code	Meaning
`0`	A digit placeholder that always shows, even as a padding zero (`0000` displays `5` as `0005`).
`#`	A digit placeholder that shows *only if significant* (no forced padding zero).
`,`	Thousands separator when placed between digit placeholders; when placed at the *end* of the code, it scales the display down by one thousand per comma without changing the stored value.
`.`	Decimal point.
`%`	Multiplies by 100 and appends a percent sign.
`"text"`	Literal text inserted verbatim into the display.
`[Red]`, `[Blue]`, `[Green]`, etc.	Colour tag placed at the start of a section — one of eight named colours available without VBA.
`@`	Placeholder for the original text entry (used only in the fourth, text, section).

Worked examples, exactly as typed into the Custom Type box:

Format code	Input value	Displayed as
`#,##0.00`	`1234.5`	`1,234.50`
`$#,##0.00;[Red]($#,##0.00)`	`-1234.5`	`($1,234.50)` in red
`#,##0.00;(#,##0.00);"-"`	`0`	`-` (a dash instead of `0.00`)
`0.0,,"M"`	`5000000`	`5.0M`
`0.0,"K"`	`45000`	`45.0K`
`0.0%`	`0.1235`	`12.3%`

Why This Matters in Modeling

Investment banking and equity research models near-universally use two custom conventions built from the codes above: (1) **negative numbers in red parentheses** rather than a minus sign, because a minus sign is easy to miss at a glance in a dense grid, and (2) **abbreviating large figures to K/M/Bn** (`0.0,,"M"`) so a model showing enterprise values in the billions doesn't force nine-digit numbers into every cell.

4.3 Text Numbers: A Silent Data-Quality Trap

Numbers Stored as Text

When a number is imported from a CSV, a bank statement export, or typed with a leading apostrophe, Excel may store it as a *text string* that merely *looks* like a number (left-aligned by default, versus right-aligned for true numbers, often flagged with a small green corner triangle). Such cells will silently **fail to be included** in a `SUM` or `AVERAGE` range, with no error message — the total is simply, quietly wrong. Always check alignment (numbers right-align by default) when a total looks suspicious. The fastest fix: select the range, use **Data** → **Text to Columns** → **Finish** (this forces a re-parse of the values), or multiply by 1 via a helper formula, or use **Paste Special** → **Multiply** with a `1` pasted from any cell.

5 Formula Fundamentals

5.1 Anatomy of a Formula

Every formula begins with `=` (a leading `+` is also silently accepted, a holdover from Lotus 1-2-3 compatibility). Everything after that is evaluated as an expression combining constants, cell references, operators, and functions.

Arithmetic Operators

Operator	Meaning	Operator	Meaning
`+`	Addition	`^`	Exponentiation
`-`	Subtraction	`&`	Text concatenation
`*`	Multiplication	`=, <>, >, <, >=, <=`	Comparison (returns `TRUE`/`FALSE`)
`/`	Division		

Order of operations follows standard mathematical precedence (PEMDAS-equivalent): parentheses first, then exponentiation, then multiplication/division (left to right), then addition/subtraction (left to right). As in any programming language, **use parentheses liberally** rather than relying on memorised precedence rules — it costs nothing and prevents an entire class of subtle model errors.

Formula	Result	Why
`=2+3*4`	`14`	Multiplication before addition
`=(2+3)*4`	`20`	Parentheses override default precedence
`=10/2^2`	`2.5`	Exponentiation before division

5.2 Cell References — A First Look

A formula almost never contains raw constants only; it references other cells so that the calculation stays live. Typing `=B4+B5` in cell `B6` tells Excel "add whatever is currently in B4 to whatever is currently in B5." Change B4, and B6 recalculates instantly — this reactive recalculation is the entire reason Excel is used for modeling instead of a plain calculator.

Key Concept

By default every reference you type (e.g. `B4`) is a **relative reference**: if that formula is copied to another cell, Excel shifts the reference by the same offset as the copy. This is usually exactly what you want when copying a formula down a column of similar rows, and exactly what you *don't* want when a formula must always point at one fixed cell (e.g., a tax rate assumption cell referenced by every row). The fix — **absolute references** using the `$` symbol (`$B$4`) — is a full topic in its own right and is covered in depth in Module 2, since it interacts closely with Named Ranges. For now, just recognise that this distinction exists and watch for formulas that "break" in a subtly wrong way after being copied.

5.3 Starter Functions Every Analyst Uses Daily

A **function** is a named, pre-built formula that takes **arguments** in parentheses, separated by commas, and returns a value: `=FUNCTION(argument1, argument2, ...)`.

SUM

`=SUM(number1, [number2], ...)` — adds every number in the given range(s). Ranges and individual cells can be mixed: `=SUM(B4:B8, D2)`.

AVERAGE

`=AVERAGE(number1, [number2], ...)` — the arithmetic mean of the given values, automatically ignoring blank cells (but *not* ignoring zeros, which will pull the average down — a frequent source of confusion versus `AVERAGEIF`, covered in Module 3).

COUNT and COUNTA

`=COUNT(value1, ...)` counts only cells containing **numbers**. `=COUNTA(value1, ...)` counts **any non-blank** cell, numeric or text. Mixing these up is a common bug when counting how many transactions/entries exist in a column that includes text labels.

MIN and MAX

`=MIN(number1, ...)` / `=MAX(number1, ...)` return the smallest/largest value in the range — e.g., the lowest and highest daily closing price in a range of stock prices.

TODAY and NOW

`=TODAY()` returns the current date (no time component); `=NOW()` returns current date *and* time. Both take no arguments but the parentheses are still required. Both recalculate every time the workbook opens or recalculates — useful for "as-of" labels on reports, but remember they are *not static*: a model saved today will show a different `TODAY()` value if reopened next month.

ROUND

`=ROUND(number, num_digits)` rounds to a specified number of decimal places (`num_digits` can be negative to round to tens/hundreds/thousands, e.g. `=ROUND(1234, -2)` returns `1200`). Note this is distinct from *display* rounding via number formatting (§4) — `ROUND` changes the actual stored value, which matters when that value feeds into further calculations.

AutoSum and the Fill Handle

`Alt+=` (AutoSum) automatically inserts a `SUM` formula guessing the adjacent range — the fastest way to total a column. The **Fill Handle** (the small square at a selected cell's bottom-right corner) lets you drag a formula across adjacent cells, automatically adjusting relative references as it goes; double-clicking it auto-fills down to match the length of an adjacent populated column.

5.4 Seeing the Formulas, Not Just the Results

`Ctrl+‘` (grave accent, top-left key) toggles **Show Formulas** mode, displaying every cell's underlying formula instead of its calculated result across the whole sheet at once — indispensable for auditing someone else's model or debugging your own. **Formulas → Trace Precedents** / **Trace Dependents** draws arrows showing which cells feed into (precedents) or depend on (dependents) the selected cell, and **Formulas → Evaluate Formula** steps through a complex nested formula one calculation at a time, similar to a debugger's step-through execution.

6 Keyboard Shortcuts for Speed

Shortcuts are not a cosmetic convenience in Excel — a fluent analyst who never touches the mouse can build and audit a model several times faster than one who doesn't, and it is a visible marker of proficiency in interviews and on the job. All shortcuts below are Windows; on macOS, substitute `Cmd` for `Ctrl` in most (not all) cases, noted where it differs.

6.1 Navigation

Shortcut	Action
`Ctrl + Arrow key`	Jump to the edge of the current data region in that direction
`Ctrl + Home`	Jump to cell `A1`
`Ctrl + End`	Jump to the last used cell on the sheet
`Page Up / Page Down`	Scroll one screen up/down
`Alt + Page Up / Down`	Scroll one screen left/right
`Ctrl + Page Up / Down`	Move to the previous/next worksheet tab
`F5` or `Ctrl+G`	Open "Go To" — jump directly to a typed cell address

6.2 Selection

Shortcut	Action
`Ctrl + Shift + Arrow key`	Extend selection to the edge of the current data region
`Ctrl + A`	Select the entire current data region (press twice for the whole sheet)
`Shift + Space`	Select the entire current row
`Ctrl + Space`	Select the entire current column
`Ctrl + Click`	Add a non-adjacent cell/range to the current selection

6.3 Editing

Shortcut	Action
`Ctrl + X / C / V`	Cut / Copy / Paste
`Ctrl + Alt + V`	Paste Special (values only, formulas only, formats only, transpose, ...)
`Ctrl + Z / Y`	Undo / Redo
`Ctrl + D`	Fill down (copy the top cell of a selection into all cells below it)
`Ctrl + R`	Fill right
`F2`	Edit the active cell in place
`Delete`	Clear cell contents (keeps formatting)
`Ctrl + -` / `Ctrl + Shift + +`	Delete / Insert selected cells, rows, or columns

6.4 Formatting

Shortcut	Action
`Ctrl + 1`	Open Format Cells dialog (the single most useful formatting shortcut)
`Ctrl + B / I / U`	Bold / Italic / Underline
`Ctrl + Shift + $`	Apply Currency format
`Ctrl + Shift + %`	Apply Percentage format
`Ctrl + Shift + #`	Apply Date format
`Ctrl + Shift + 1`	Apply Number format (two decimals, thousands separator)

6.5 Formulas & Calculation

Shortcut	Action
`Alt + =`	AutoSum
`F4` (while editing a reference)	Cycle a reference through relative/absolute combinations (see Module 2)
`Ctrl + ‘`	Toggle Show Formulas view
`F9`	Recalculate the entire workbook
`Shift + F9`	Recalculate only the active sheet

6.6 File & View

Shortcut	Action
`Ctrl + N / O / S`	New / Open / Save
`F12`	Save As
`Ctrl + P`	Print
`Ctrl + F / H`	Find / Replace (full depth in Module 3)
`Alt + Tab`	Switch open windows/applications

7 Cross-Tool Translation: Python

7.1 Why Two Libraries, Not One

Python has no single "Excel library" because Excel actually does two separate jobs that Python keeps cleanly separated:

- `pandas` — for *data*: loading, filtering, aggregating, and computing over tabular data efficiently, the equivalent of what a formula-filled range of cells *does*.
- `openpyxl` — for the *file*: reading and writing actual `.xlsx` files cell-by-cell, including fonts, fills, number formats, and even literal formula strings — the equivalent of what the worksheet *looks like*.

In practice, a typical script uses `pandas` to do the heavy numerical lifting and calls `openpyxl` (often via `pandas.to_excel`, which uses `openpyxl` internally) only at the final step, to produce a polished, formatted `.xlsx` deliverable.

Listing 1: Environment setup

```
pip install pandas openpyxl
```

7.2 Workbook and Worksheet Equivalents (openpyxl)

Listing 2: Creating a workbook, equivalent to File → New → Blank Workbook

```
from openpyxl import Workbook

wb = Workbook()                  # a new Workbook, like a blank .xlsx
ws = wb.active                   # the first worksheet, like "Sheet1"
ws.title = "Monthly Expenses"    # renaming the sheet tab

# Writing to specific cells, exactly like typing into the grid
ws["A1"] = "Category"
ws["B1"] = "Amount"
ws["A2"] = "Rent"
ws["B2"] = 25000

# Cell addresses work exactly as in Excel: column letter + row number
print(ws["B2"].value)            # 25000

wb.save("expenses.xlsx")         # equivalent of Ctrl+S
```

Listing 3: Opening an existing workbook, equivalent to File → Open

```
from openpyxl import load_workbook

wb = load_workbook("expenses.xlsx")
print(wb.sheetnames)             # list of worksheet tab names, like the
    tabs at the bottom
ws = wb["Monthly Expenses"]      # select a sheet by its tab name
```

7.3 Reading a Whole Sheet as a DataFrame (pandas)

Where `openpyxl` operates cell-by-cell, `pandas` treats a whole worksheet range as one table — much closer to how you *think* about a dataset in finance.

Listing 4: Reading an Excel sheet into pandas

```
import pandas as pd
```

```

df = pd.read_excel("expenses.xlsx", sheet_name="Monthly Expenses")
print(df.head())

#      Category   Amount
# 0        Rent    25000
# 1        Food     8000
# 2   Transport     3000
```

7.4 Number Formatting (openpyxl)

This is the single most direct Excel↔Python bridge in this whole module: `openpyxl` accepts *the exact same custom format code strings* you would type into Excel's Format Cells dialog (§4.2).

Listing 5: Applying Accounting/Currency format codes identical to Excel's

```
from openpyxl.styles import Font, PatternFill, Alignment

cell = ws["B2"]

# Same custom format code as typed into Excel's Custom Type box
cell.number_format = '$#,##0.00;[Red](#,##0.00)'

# Font styling -- equivalent of Home tab Font group
cell.font = Font(name="Calibri", size=11, bold=True, color="1F5C8B")

# Fill colour -- equivalent of Home tab Fill Colour
cell.fill = PatternFill(start_color="E9F3EE", end_color="E9F3EE",
    fill_type="solid")

# Alignment -- equivalent of Home tab Alignment group
cell.alignment = Alignment(horizontal="right", vertical="center",
    wrap_text=True)

wb.save("expenses.xlsx")
```

7.5 Aggregate Formulas: pandas vs. Writing Live Excel Formulas

There are two distinct ways Python can "do a SUM," and the difference matters:

Listing 6: Option A: compute the result in Python (a static number)

```
total = df["Amount"].sum()          # equivalent of =SUM(), computed once
     in Python
average = df["Amount"].mean()       # equivalent of =AVERAGE()
count_rows = df["Amount"].count()   # equivalent of =COUNT() (numeric
    cells only)
minimum = df["Amount"].min()        # equivalent of =MIN()
maximum = df["Amount"].max()        # equivalent of =MAX()
```

Listing 7: Option B: write an actual live Excel formula into the .xlsx file

```
# openpyxl can write the formula STRING itself, so it stays "live"
# and recalculates in Excel if the source cells later change
ws["B10"] = "=SUM(B2:B9)"
ws["B11"] = "=AVERAGE(B2:B9)"
wb.save("expenses.xlsx")
```

Key Concept

Option A produces a *frozen* number computed at script run-time — fast and simple, but the value goes stale the moment the source data changes, exactly like a Paste-Values operation in Excel. Option B produces a *live formula string* inside the `.xlsx` file that Excel itself will recalculate when opened — slower to set up, but it preserves the "reactive" property that makes Excel useful in the first place. Choose Option B whenever the output workbook needs to remain editable and self-updating for a human user; choose Option A when Python is just generating a final static report.

7.6 Saving and Exporting

Listing 8: Common export paths, mirroring Excel's Save/Save As/Export

```
wb.save("expenses.xlsx")                          # equivalent of Ctrl+S / Save As .xlsx
df.to_excel("expenses_export.xlsx", index=False)   # pandas' own writer (uses openpyxl internally)
df.to_csv("expenses.csv", index=False)             # equivalent of Save As -> CSV (values only)
```

8 Cross-Tool Translation: SQL

8.1 The Mental Model

Excel	SQL	Notes
Workbook	Database	A container for multiple tables/sheets.
Worksheet	Table	A named grid of rows and columns.
Column header	Column name	In SQL, a name is mandatory and fixed at table creation; in Excel a header is just a text cell you type.
Row	Record/Row	A single entity's set of values.
Cell	Field	One value at the intersection of a row and column.
Number Format	(No direct equivalent)	SQL stores a value's raw **data type** (`DECIMAL`, `DATE`, etc.); *how* it is displayed to a user is handled entirely outside the database, at the application/reporting layer — exactly like the Excel principle from §4 that format is cosmetic and separate from stored value, just pushed one layer further away.

Key Concept

The biggest conceptual shift moving from Excel to SQL is that a SQL table has **no inherent row order and no concept of a cell address**. There is no `C18` in SQL — you always retrieve and identify rows by the *values* in named columns (e.g., `WHERE category = 'Rent'`), never by position. This is the same relative/absolute-reference tension flagged in §5: SQL sidesteps the whole problem by refusing to have positional references at all.

8.2 Defining the Table (equivalent of setting up columns & formats)

Declaring a SQL table's column **data types** is the closest SQL equivalent to choosing Excel's Number Format — both are decisions about what kind of value a column is allowed to hold.

Listing 9: CREATE TABLE, equivalent to typing headers and setting column formats

```
CREATE TABLE monthly_expenses (
    expense_id   INT PRIMARY KEY,
    category     VARCHAR(50)    NOT NULL,    -- like a Text-formatted
        column
    amount       DECIMAL(12,2)  NOT NULL,    -- like a Currency/Accounting
        -formatted column
    expense_date DATE           NOT NULL     -- like a Date-formatted
        column
);
```

Finance Application

`DECIMAL(12,2)` rather than a floating-point type is the standard choice for money columns in real financial systems: it stores an exact fixed-point value (12 total digits, 2 after the decimal), avoiding the tiny binary floating-point rounding errors that can silently accumulate across millions of transactions – a class of bug Excel does not fully protect you from either, which is why `ROUND` (§5) matters.

8.3 Inserting Data (equivalent of typing values into cells)

Listing 10: INSERT, equivalent to typing rows of data into a worksheet

```
INSERT INTO monthly_expenses (expense_id, category, amount, expense_date)
    VALUES
    (1, 'Rent',          25000.00, '2026-08-01'),
    (2, 'Food',            8000.00, '2026-08-03'),
    (3, 'Transport',       3000.00, '2026-08-05'),
    (4, 'Utilities',       4500.00, '2026-08-07'),
    (5, 'Entertainment', 2000.00, '2026-08-10');
```

8.4 Aggregate Queries (equivalent of SUM, AVERAGE, COUNT, MIN, MAX)

SQL's built-in aggregate functions map almost one-to-one onto the Excel functions from §5.

Excel	SQL	Notes
`SUM`	`SUM()`	Identical concept.
`AVERAGE`	`AVG()`	Identical concept.
`COUNT`	`COUNT(column)`	Counts non-null values in a column, closer to Excel's `COUNT` (numeric-aware) than `COUNTA`.
`COUNTA`	`COUNT(*)`	Counts every row regardless of null fields.
`MIN / MAX`	`MIN() / MAX()`	Identical concept.

Listing 11: Aggregate query, equivalent to a small block of SUM/AVERAGE/COUNT/MIN/MAX formulas

```
SELECT
    SUM(amount)                  AS total_expenses,    -- =SUM(B2:B6)
```

```sql
    ROUND(AVG(amount), 2)        AS average_expense,    -- =AVERAGE(B2:B6)
    COUNT(*)                     AS number_of_entries,  -- =COUNTA(A2:A6)
    MIN(amount)                  AS smallest_expense,   -- =MIN(B2:B6)
    MAX(amount)                  AS largest_expense     -- =MAX(B2:B6)
FROM monthly_expenses;
```

9 Worked Example: Monthly Expense Summary, Three Ways

The same small dataset — five expense categories — built out identically in all three tools, to make the equivalence concrete rather than abstract.

9.1 The Data

Category	Amount (INR)
Rent	25,000
Food	8,000
Transport	3,000
Utilities	4,500
Entertainment	2,000
Total	**42,500**

9.2 Excel Implementation

- Categories in `A2:A6`, amounts in `B2:B6`, both formatted with the **Accounting** number format (§4).
- `B7`: `=SUM(B2:B6)` → `42,500`, with a top border applied for a subtotal line (a formatting convention, not a formula).
- `B8`: `=AVERAGE(B2:B6)` → `8,500`.
- Column `A` header (`A1:B1`) is Bold, with Fill Colour applied, following the blue/black colour convention from §4 for the two hard-coded input columns.

9.3 Python Implementation

Listing 12: End-to-end: build the data, compute totals, write a formatted .xlsx

```python
import pandas as pd
from openpyxl import Workbook
from openpyxl.styles import Font, Alignment, Border, Side

data = {
    "Category": ["Rent", "Food", "Transport", "Utilities", "Entertainment
        "],
    "Amount":   [25000, 8000, 3000, 4500, 2000],
}
df = pd.DataFrame(data)

total = df["Amount"].sum()           # 42500
average = df["Amount"].mean()        # 8500.0

wb = Workbook()
ws = wb.active
ws.title = "Expenses"
```

```
ws.append(["Category", "Amount"])
for _, row in df.iterrows():
    ws.append([row["Category"], row["Amount"]])

# Live formulas, matching the Excel version cell-for-cell
ws["B7"] = "=SUM(B2:B6)"
ws["B8"] = "=AVERAGE(B2:B6)"

for row in ws["B2:B8"]:
    for cell in row:
        cell.number_format = '#,##0'

ws["B7"].border = Border(top=Side(style="thin"))
ws["A1"].font = ws["B1"].font = Font(bold=True)

wb.save("expenses_summary.xlsx")
```

9.4 SQL Implementation

Listing 13: Same summary via schema + aggregate query

```
CREATE TABLE monthly_expenses (
    category VARCHAR(50) PRIMARY KEY,
    amount   DECIMAL(10,2) NOT NULL
);

INSERT INTO monthly_expenses VALUES
    ('Rent',          25000.00),
    ('Food',           8000.00),
    ('Transport',      3000.00),
    ('Utilities',      4500.00),
    ('Entertainment',  2000.00);

SELECT
    SUM(amount)           AS total,      -- 42500.00
    ROUND(AVG(amount), 2) AS average     -- 8500.00
FROM monthly_expenses;
```

10 Common Pitfalls & Debugging Checklist

Symptom	Likely cause & fix
Column shows `#####`	Column too narrow for the formatted value — widen the column (double-click the column border to auto-fit); the underlying value is fine.
`SUM` total looks too low	Some "numbers" in the range are actually text (§4) — check for left-aligned entries in an otherwise right-aligned numeric column.
Sorting or filtering scrambles data	Merged cells somewhere in the range (§3.3.2) — unmerge before sorting/filtering.
Formula gives a wildly wrong result after copying	A reference that should have stayed fixed shifted because it was relative, not absolute — revisit §5 and Module 2's `$` notation.
A formula references its own cell, directly or indirectly	**Circular reference** warning — Excel will flag this explicitly; trace the chain with **Formulas → Error Checking → Circular References**.
Numbers display differently for a colleague in another country	Regional settings differ: some locales use `,` as the decimal separator and `.` as the thousands separator (the reverse of the US/India convention used in this handbook) — the stored value is identical, only the display convention differs.

11 Chapter Summary & Self-Check

Before moving to Module 2 (Optimisation, Macros, Absolute/Relative References, Named Ranges, Data Validation & Custom Formatting), you should be able to, without looking anything up:

- ☐ Explain the difference between a workbook, a worksheet, and a cell, and give the pandas/SQL equivalent of each.
- ☐ Navigate the File tab confidently: New vs. Open, Save vs. Save As, and when to export to PDF vs. CSV.
- ☐ Explain *why* merging cells inside a data table is dangerous, and what to use instead for a visually centered title.
- ☐ State the difference between Currency and Accounting number formats, and write a custom format code that shows negative numbers in red parentheses.
- ☐ Write `SUM`, `AVERAGE`, `COUNT`/`COUNTA`, `MIN`, `MAX`, `TODAY`, and `ROUND` from memory, including correct argument syntax.
- ☐ Recognise the symptom of "numbers stored as text" and know at least one fix.
- ☐ Freeze panes on a large sheet without looking up the steps.
- ☐ Recall at least 10 keyboard shortcuts across navigation, editing, and formatting without a reference sheet.
- ☐ Write the `openpyxl` code to create a workbook, write values, and apply a custom number format identical to an Excel one.
- ☐ Write a `CREATE TABLE` statement choosing appropriate data types for a small financial dataset, and an aggregate `SELECT` equivalent to a block of Excel summary formulas.

12 What's Next: Module 2 Preview

Module 2, *Making Excel Professional*, builds directly on this foundation and covers:

- **Optimisation** — techniques for keeping large, formula-heavy workbooks fast and maintainable.
- **Macros** — recording and writing VBA to automate repetitive Excel tasks, with a comparison to automating the same tasks in Python.
- **Absolute vs. Relative References** — the full `$` notation this module deliberately deferred, including mixed references and the `F4` cycling shortcut.
- **Named Ranges** — replacing fragile cell-address references (`B4`) with stable, self-documenting names (`TaxRate`), directly solving the positional-reference fragility flagged in §3.1.
- **Data Validation & Custom Formatting** — constraining what a user is allowed to type into a cell, the closest Excel gets to a SQL `CHECK` constraint or a Python input-validation function.